

1. 자료형

- 1-1 숫자형
- 1-2 문자열 자료형
- 1-3 리스트 자료형
- 1-4 튜플 자료형
- 1-5 딕셔너리 자료형
- 1-6 집합 자료형
- 1-7 불 자료형
- 1-8 변수와 메모리



■ 숫자형(Number)

- 정수, 실수, 8진수, 16진수 등

정수형 (Integer)

```
a = 42
```

```
a = -7
```

```
a = 0
```

```
print(type(a))    # <class 'int'>
```

실수형 (Floating-point)

```
a = 3.14
```

```
a = -0.5
```

```
print(type(a))    # <class 'float'>
```

지수 표기법 (Scientific notation)

```
a = 6.02e23      # 6.02 × 1023
```

```
a = 1.6e-19      # 1.6 × 10-19
```

8진수 (Octal)

```
a = 0o52         # octal → decimal 42
```

16진수 (Hexadecimal)

```
a = 0x2A         # hexadecimal → decimal 42
```

```
a = 0xFF         # hexadecimal → decimal 255
```

1-1. 숫자형



■ 산술 연산자

- 사칙연산(+, -, *, /), 제곱(**), 나머지(%), 몫(//)

```
a = 6  
b = 4
```

사칙 연산

```
a + b    # 10  
a - b    # 2  
a * b    # 24  
a / b    # 1.5
```

제곱 연산자

```
2 ** 5   # 32  
5 ** 2   # 25
```

나머지 연산자

```
7 % 3    # 1  
3 % 7    # 3
```

몫 연산자

```
7 / 4    # 1.75  
7 // 4   # 1
```



■ 복합 연산자

- 산술 연산과 대입을 한 번에 처리하는 방법

기본 대입

a = 3

일반적인 증가

a = a + 2

복합 연산자

a += 2

복합 연산자 목록

a += 2 # a = a + 2

a -= 1 # a = a - 1

a *= 3 # a = a * 3

a /= 2 # a = a / 2

a //= 2 # a = a // 2

a %= 2 # a = a % 2

a **= 2 # a = a ** 2

1-1. 숫자형



■ 산술 연산자 우선순위

- 여러 연산자가 함께 사용될 때 계산되는 순서

우선순위	연산자	설명
1	()	괄호 (가장 먼저 계산)
2	**	거듭제곱
3	*, /, //, %	곱셈, 나눗셈, 몫, 나머지
4	+, -	덧셈, 뺄셈

```
print(2 + 3 * 4)           # 14
print((2 + 3) * 4)         # 20

print(2 ** 3 * 2)          # 16
print(2 ** (3 * 2))        # 64

print(10 - 4 / 2)          # 8.0
```

1-2. 문자열 자료형



- 문자열(String) 생성
 - 문자열은 따옴표(' ', " ")로 둘러싼다.

```
a = "Hello World"  
b = 'Python is fun'
```

```
# 따옴표 3개를 사용한 여러 줄 문자열  
(multiline string)
```

```
c = """Life is too short,  
You need python."""
```

```
d = '''  
오늘부터 파이썬 공부 시작  
아자아자 파이팅!  
'''
```

```
# 문장에 작은따옴표 포함
```

```
a = "It's time to study Python"
```

```
# 문장에 큰따옴표 포함
```

```
b = '"Python is very easy." he says.'
```

```
# 따옴표를 문자로 사용하기
```

```
c = 'It\'s time to study Python'
```

```
d = "\"Python is very easy.\" he says."
```

1-2. 문자열 자료형



■ 이스케이프 코드(Escape Codes)

- 문자열 안에서 줄바꿈이나 탭과 같은 특수 동작을 표현하기 위해 사용하는 문자 조합

\n	줄 바꿈
\t	탭 간격
\\	\ 문자 출력
\'	작은따옴표 출력
\"	큰따옴표 출력

줄 바꿈

```
print("Hello\nPython")
```

탭

```
print("Name:\tAlice")
```

역슬래시

```
print("C:\\Users\\python")
```


1-2. 문자열 자료형



■ 문자열 연산과 길이

■ 문자열 더하기 (연결)

```
a = "Study"  
b = " Python"  
  
a + b
```

```
>>> Study Python
```

■ 문자열 곱하기 (반복)

```
a = "Go! "  
  
a * 3
```

```
>>> Go! Go! Go!
```

■ 문자열 길이 구하기

```
a = "Keep going"  
  
len(a)
```

```
>>> 10
```

1-2. 문자열 자료형



■ 문자열 인덱싱

- 인덱싱(indexing): 문자열에서 특정 위치의 문자 하나를 가리켜 꺼내는 것
- 파이썬은 항상 0부터 번호를 센다

```
text = "Python is fun."
```

P	y	t	h	o	n		i	s		f	u	n	.
0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
text[0]    # 'P'  
text[3]    # 'h'  
text[7]    # 'i'
```

```
# 뒤에서 부터 세기  
text[-1]   # '.'  
text[-3]   # 'u'  
text[-0]   # text[0]과 동일
```

1-2. 문자열 자료형



■ 문자열 슬라이싱

- 슬라이싱(slicing): 문자열에서 여러 문자를 한 번에 잘라내는 방법
- 인덱싱이 "하나를 집는다"면, 슬라이싱은 "범위를 잘라낸다"

```
text = "Python is fun."
```

P	y	t	h	o	n		i	s		f	u	n	.
0	1	2	3	4	5	6	7	8	9	10	11	12	13

■ 기본 슬라이싱 문법

```
text[시작번호:끝번호:간격]  
# 시작번호 <= index < 끝번호  
# 간격(step): 몇 칸씩 건너뛴지 (기본값 1)
```

■ 기본 슬라이싱 예제

```
text[0:7]    # 'Python '  
text[7:9]    # 'is'  
text[1:6:2]  # 'yhn'
```

1-2. 문자열 자료형



■ 문자열 슬라이싱 활용

```
text = "Python is fun."
```

시작/끝 생략하기

```
text[:6]      # 'Python'
text[7:]      # 'is fun.'
text[:]       # 'Python is fun.'
```

```
text[-4:]     # 'fun.'
text[7:-5]    # 'is'
```

문자열 순서 뒤집기

```
text[::-1]    # '.nuf si nohtyP'
```

■ 예제 1. 날짜 정보 나누기

```
today = "20260224"
```

```
year = today[:4]    # '2026'
month = today[4:6]  # '02'
date = today[6:]    # '24'
```

■ 예제 2. 오타 수정하기

```
word = "Pythin"
```

```
word[:4] + 'o' + word[5:]  # 'Python'
```

■ 문자열 포매팅(String Formatting)

- 문자열의 일정 부분만 값을 바꿔 넣는 방법

고정된 문자열

```
print("I studied for 4 hours.")
```

정수 대입

```
print("I studied for %d hours." % 5)
```



실수 대입

```
print("I studied for %f hours." % 5.5)
```

문자열 대입

```
print("I studied for %s hours." % "five")
```

숫자 값을 나타내는 변수로 대입

```
hours = 5
```

```
print("I studied for %d hours." % hours)
```

2개 이상의 포맷 코드 함께 사용

```
"I studied for %d hours and felt %s." % (5, "tired")
```

*모든 자료형을 문자형으로 변환

%s	문자열(String)*
%c	문자 1개(character)
%d	정수(Integer)
%f	부동소수(floating-point)
%o	8진수
%x	16진수
%%	Literal % (문자 % 자체)

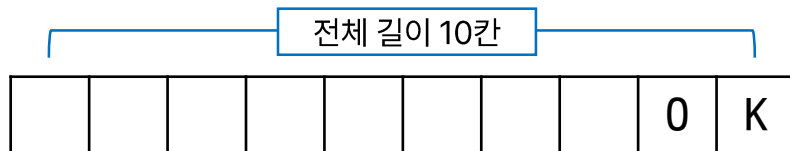
1-2. 문자열 자료형



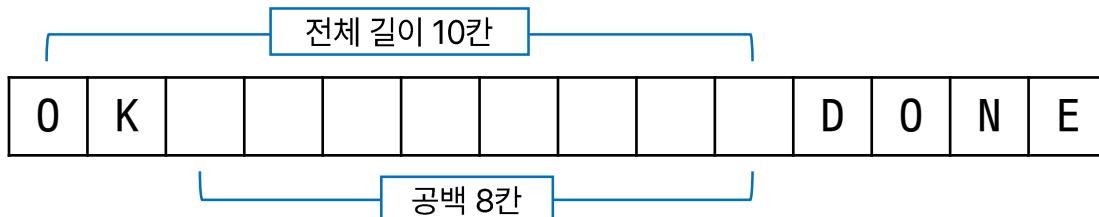
■ 문자열 포매팅 출력 형식 제어

- 포맷 코드에 숫자를 함께 사용하면 출력되는 문자열의 정렬, 자릿수, 표시 형식을 조절할 수 있다

```
# 전체 폭 10, 오른쪽 정렬  
print("%10s" % "OK")
```



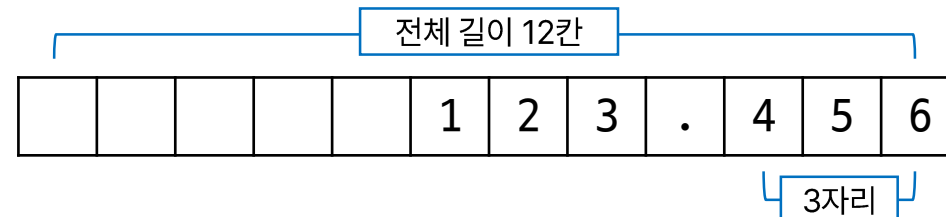
```
# 전체 폭 10, 왼쪽 정렬  
print("%-10sDONE" % "OK")
```



```
# 소수점 자리수 지정(반올림)  
rate = 87.456  
print("%.2f" % rate)
```

```
# 퍼센트(%) 함께 출력  
print("Success rate: %.1f%%" % rate)
```

```
# 전체 폭 12, 소수점 이하 3자리  
print("%.3f" % 123.45678)
```





■ 문자열 포매팅 - format() 함수 사용

- % 연산자 대신 문자열의 format() 함수를 사용
- 값이 들어갈 자리를 {}로 표시

순서 기반 포매팅

```
print("I studied for {} hours and felt {}".format(5, "tired"))
```

인덱스 기반 포매팅

```
print("I studied for {0} hours and felt {1}".format(5, "tired"))
```

이름 기반 포매팅

```
print("I studied for {hours} hours and felt {feeling}".format(hours=5, feeling="tired"))
```

인덱스 + 이름 혼용

```
print("I studied for {0} hours and felt {feeling}".format(5, feeling="tired"))
```

중괄호{} 문자 출력

```
print("Result: {{x = {}}}".format(10))
```

1-2. 문자열 자료형



■ 문자열 포매팅 - format() 함수 사용

- % 연산자 대신 문자열의 format() 함수를 사용
- 값이 들어갈 자리를 {}로 표시

오른쪽 정렬 (폭 10)

```
print("{:>10}".format("OK"))
```

왼쪽 정렬 (폭 10)

```
print("{:<10}DONE".format("OK"))
```

소수점 자리수 지정

```
rate = 87.456
```

```
print("{:.2f}".format(rate))
```

퍼센트 출력 (% 불필요)

```
print("Success rate: {:.1f}%".format(rate))
```

■ % 포매팅에는 없던 새로운 기능

가운데 정렬

```
print("{:^10}".format("OK"))
```

문자로 채우기

```
print("{:=^10}".format("OK"))
```

```
print("{:!10".format("OK"))
```

=	=	=	=	0	K	=	=	=	=
---	---	---	---	---	---	---	---	---	---

0	K	!	!	!	!	!	!	!	!
---	---	---	---	---	---	---	---	---	---

■ f 문자열 포매팅(f-string)

- 문자열 앞에 f를 붙이고 {} 안에 변수, 표현식을 바로 작성 가능
- Python 3.6 이상에서 사용 가능

변수 사용

```
hours = 5
```

```
feeling = "tired"
```

```
print(f"I studied for {hours} hours and felt {feeling}.")
```

표현식 사용

```
age = 29
```

```
print(f"Next year, I will be {age + 1}.")
```

중괄호{} 문자 출력

```
print(f"Result: {{x = {10}}}")
```

■ f 문자열 포매팅(f-string)

- 문자열 앞에 f를 붙이고 {} 안에 변수, 표현식을 바로 작성 가능
- Python 3.6 이상에서 사용 가능

```
# 오른쪽 정렬 (폭 10)
```

```
print(f"{'OK':>10}")
```

```
# 왼쪽 정렬 (폭 10)
```

```
print(f"{'OK':<10}DONE")
```

```
# 소수점 자리수 지정
```

```
rate = 87.456
```

```
print(f"{rate:.2f}")
```

```
# 퍼센트 출력 (% 불필요)
```

```
print(f"Success rate: {rate:.1f}%")
```

■ f-string에만 있는 기능

```
# 숫자에 콤마(,) 자동 삽입
```

```
print(f"총 금액은 {119000:,.}원입니다.")
```

총 금액은 119,000원입니다.

1-2. 문자열 자료형



■ 문자열 내장 함수

- 문자열 변수 이름 뒤에 .를 붙이고 함수 이름을 호출하여 사용한다.

count()	문자 개수 세기
find()	위치 찾기
index()	위치 찾기
join()	문자열 삽입
split()	문자열 분리
upper()	대문자로 변환
lower()	소문자로 변환
capitalize()	첫 글자 대문자로 변환

replace()	문자열 치환
strip()	양쪽 공백 제거
lstrip()	왼쪽 공백 제거
rstrip()	오른쪽 공백 제거
isalpha()	알파벳 여부
isdigit()	숫자 여부
startswith()	시작 검사
endswith()	끝 검사

1-2. 문자열 자료형



■ 문자열 내장 함수 예제

```
# 개수 / 위치
a = "banana"
a.count('a')          # 3

b = "Error: file not found"
b.find("file")        # 7
b.find("success")     # -1
b.index("success")    # ValueError

# 문자열 변형
a = " hello "
a.strip()             # 'hello'
a.lstrip()            # 'hello '

b = "Hello Python"
b.upper()             # 'HELLO PYTHON'
b.lower()             # 'hello python'
```

```
# 분리 / 결합
print(", ".join("1234"))

a = "red:green:blue"
print(a.split(":"))

# 형태 검사
a, b, c = "Python", "0620", "F1"
a.isalpha()           # True
b.isdigit()           # True

# 시작 / 끝 검사
a = "report.pdf"
a.endswith(".pdf")    # True

a = "https://example.com"
a.startswith("https") # True
```

■ 리스트 자료형

- 리스트는 여러 개의 데이터를 순서대로 저장하고 한 번에 관리할 수 있는 자료형이다.
- 대괄호[]로 만들고 요소는 쉼표로 구분한다.

```
리스트명 = [요소1, 요소2, 요소3, ...]
```

```
scores = [85, 90, 78, 92]
words = ["Python", "is", "fun"]
mixed = [1, "hello", 3.14]
nested = [1, 2, ["a", "b", "c"]] # 중첩 리스트

# 빈 리스트 생성
empty1 = []
empty2 = list()
```

■ 리스트의 인덱싱

- 리스트는 순서가 있는 자료형이므로, 인덱스를 이용해 특정 요소를 선택할 수 있다.

기본 인덱싱

```
nums = [4, 7, 1]
```

```
nums[0]          # 4
```

음수 인덱싱

```
nums[-1]         # 1
```

인덱스를 활용한 연산

```
nums[0] + nums[2] # 5
```

중첩 리스트 인덱싱

```
data = [10, 20, ['x', 'y', 'z']]
```

```
data[2]          # ['x', 'y', 'z']
```

```
data[2][0]       # 'x'
```

다중 중첩 인덱싱

```
items = [1, ['A', ['apple', 'banana']]]
```

```
items[1][1][0]   # 'apple'
```



■ 리스트의 슬라이싱

- 슬라이싱을 사용하면 리스트에서 여러 개의 요소를 한 번에 가져올 수 있다.

기본 슬라이싱

```
a = [10, 20, 30, 40, 50]
```

```
a[1:4]      # [20, 30, 40]
```

앞/뒤 생략

```
a[:3]       # [10, 20, 30]
```

```
a[2:]       # [30, 40, 50]
```

슬라이싱 결과는 새로운 리스트

```
b = a[2:3]  # [30]
```

```
type(b)     # list
```

중첩 리스트 슬라이싱

```
data = [1, 2, ['red', 'blue', 'green'], 3]
```

```
data[1:3]    # [2, ['red', 'blue', 'green']]
```

```
data[2][:2]  # ['red', 'blue']
```

1-3. 리스트 자료형



■ 리스트 연산

- 리스트 연산은 문자열과 거의 동일하다.

리스트 더하기 (연결)

```
x = [10, 20]
```

```
y = [30, 40]
```

```
x + y          # [10, 20, 30, 40]
```

리스트 곱하기 (반복)

```
items = ["A", "B"]
```

```
items * 2      # ['A', 'B', 'A', 'B']
```

리스트 길이 구하기

```
data = [1, 2, 3, 4]
```

```
len(data)      # 4
```

숫자와 문자열은 바로 더할 수 없다

```
nums = [5, 6, 7]
```

```
nums[1] + "kg" # TypeError
```

해결 방법: 문자열로 변환 후 더하기

```
str(nums[1]) + "kg" # '6kg'
```




■ 리스트 수정

- 리스트는 변경 가능한(mutable) 자료형이므로, 생성된 이후에도 요소의 값을 직접 바꾸거나 삭제할 수 있다.

리스트 값 수정하기

```
nums = [10, 20, 30]
```

```
nums[1] = 99      # 두 번째 요소 수정
```

```
print(nums)      # [10, 99, 30]
```

del로 리스트 요소 삭제

```
a = [80, 90, 75]
```

```
del a[0]          # 첫 번째 요소 삭제  
print(a)          # [90, 75]
```

del + 슬라이싱으로 여러 요소 삭제

```
b = [10, 20, 30, 40, 50]
```

```
del b[1:4]        # 두 번째 ~ 네 번째 요소 삭제  
print(b)          # [10, 50]
```

1-3. 리스트 자료형



■ 리스트 관련 함수

함수	기능	특징
append(x)	요소 추가	맨 뒤에 하나 추가
insert(i, x)	요소 삽입	원하는 위치에 삽입
extend(x)	리스트 확장	여러 요소를 한 번에 추가
remove(x)	요소 제거	값 기준, 첫 번째만
pop()	요소 꺼내기	삭제 + 반환
sort()	정렬	리스트 자체 변경
reverse()	뒤집기	현재 순서를 역순으로
index(x)	위치 찾기	값의 인덱스 반환
count(x)	개수 세기	값의 개수 반환



■ 리스트 관련 함수 활용

요소 추가 / 확장

```
a = [2, 4, 6]
```

```
a.append(8)
```

```
print(a)          # [2, 4, 6, 8]
```

```
a.extend([10, 12]) # 리스트를 입력
```

```
print(a)          # [2, 4, 6, 8, 10, 12]
```

```
a.insert(1, 99)    # (입력 위치, 값)
```

```
print(a)          # [2, 99, 4, 6, 8, 10, 12]
```

요소 삭제 / 꺼내기

```
a.remove(99)
```

```
print(a)          # [2, 4, 6, 8, 10, 12]
```

```
a.pop()           # 가장 마지막 값 출력 및 제거
```

```
print(a)          # [2, 4, 6, 8, 10]
```

정렬 / 순서 변경

```
b = [4, 1, 3, 2]
```

```
b.reverse()
```

```
print(b)          # [2, 3, 1, 4] # 현재 순서를 그대로 뒤집기
```

```
b.sort()
```

```
print(b)          # [1, 2, 3, 4] # 값 기준 오름차순 정렬
```

위치 / 개수 확인

```
c = [1, 2, 1, 3, 1]
```

```
print(c.index(2))  # 1
```

```
print(c.count(1))  # 3
```

■ 튜플 구조

A = ()

B = (1,)

C = (1, 2, 3)

D = ("apple", "banana", "cherry")

E = (1, "apple", 3.14, True)

■ 튜플 자료형의 특징

- 리스트와 유사하나 변경 불가능함
- 생성 시 소괄호 사용하나, 콤마만으로도 생성 가능
- 불변 특성으로 삽입/삭제/변경 불가
- 덧셈/곱셈 시 새로운 튜플이 생성됨

튜플은 요솟값을 변경할 수 없기 때문에 sort, insert, remove, pop과 같은 메서드가 없음



■ 인덱싱

```
t1 = (1, 2, 'a', 'b')  
t1[0]  
>>> 1  
t1[3]  
>>> 'b'
```

■ 슬라이싱

```
t1 = (1, 2, 'a', 'b')  
t1[1:]  
>>> (2, 'a', 'b')
```

■ 더하기

```
t1 = (1, 2, 'a', 'b')  
t2 = (3, 4)  
t3 = t1 + t2  
t3  
>>> (1, 2, 'a', 'b', 3, 4)
```

■ 곱하기

```
t2 = (3, 4)  
t3 = t2 * 3  
t3  
>>> (3, 4, 3, 4, 3, 4)
```



■ 딕셔너리 구조

```
{Key1: Value1, Key2: Value2, Key3: Value3, ...}
```

■ 딕셔너리 자료형의 특징

- Key와 Value를 한 쌍으로 가지는 자료형
- 생성 시 **중괄호** 사용, **Key: Value** 형태로 구분
- Key는 중복 불가함
- keys(), values(), items() 메소드로 데이터 추출 가능

1-5. 딕셔너리 자료형



■ 딕셔너리 예시

```
student = {'class': 'B', 'name': '김와와', 'age': '27'}
```

■ 딕셔너리 자료형 구성

Key(키)	문자열, 정수, 실수, 튜플	값이 바뀌지 않는 자료형만 가능함.
Value(값)	문자열, 숫자, 리스트, 튜플, 딕셔너리 등 모든 자료형	제한 없이 어떤 자료형이든 담을 수 있음.

1-5. 딕셔너리 자료형



■ 딕셔너리 쌍 추가

```
hobby = { 1: '축구' }  
hobby[2] = '야구'  
hobby
```

```
>>> {1: '축구', 2: '야구'}
```

```
hobby[3] = '배구'  
hobby
```

```
>>> {1: '축구', 2: '야구', 3: '배구'}
```

■ 딕셔너리 요소 삭제

```
del hobby[1]  
hobby
```

```
>>> {2: '야구', 3: '배구'}
```


1-5. 딕셔너리 자료형



■ 딕셔너리에서 Key를 사용해 Value 얻기

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict["INTP"]  
>>> 논리적인 사색가
```

```
mbti_dict["ENFP"]  
>>> 재기발랄한 활동가
```

■ 딕셔너리 주의사항

```
a = {1: 'one', 1: 'two'}
```

```
a
```

```
>>> {1: 'two'}
```

key가 중복되면, 마지막에 할당된 값으로 저장된다.

```
a = {[1, 2]: 'one two'}
```

```
>>> TypeError: unhashable type: 'list'
```

key로 리스트 사용 불가. key는 변경 불가능한 자료형이어야 한다.

1-5. 딕셔너리 자료형



■ 딕셔너리 관련 함수 – **keys** (딕셔너리의 모든 key를 반환)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.keys()
```

```
>>> dict_keys(['ENFP', 'INFP', 'ENTJ', 'INTP'])
```

```
for k in mbti_dict.keys():  
    print(k)
```

```
>>> ENFP
```

```
>>> INFP
```

```
>>> ENTJ
```

```
>>> INTP
```

1-5. 딕셔너리 자료형



■ 딕셔너리 관련 함수 – **values** (딕셔너리의 모든 value를 반환)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.values()
```

```
>>> dict_values(['재기발랄한 활동가',  
                '열정적인 중재자', '대담한 통솔자',  
                '논리적인 사색가'])
```

```
list(mbti_dict.values())
```

```
>>> ['재기발랄한 활동가', '열정적인  
중재자', '대담한 통솔자', '논리적인  
사색가']
```

1-5. 딕셔너리 자료형



■ 딕셔너리 관련 함수 – **items** (딕셔너리의 요소 쌍을 튜플 형태로 반환)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.items()
```

```
>>> dict_items([('ENFP', '재기발랄한 활동가'), ('INFP', '열정적인 중재자'), ('ENTJ',  
'대담한 통솔자'), ('INTP', '논리적인 사색가')])
```

■ 딕셔너리 관련 함수 – **clear** (딕셔너리의 모든 요소를 삭제)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.clear()  
mbti_dict
```

```
>>> {}
```

▪ 딕셔너리 관련 함수 – **get** (key에 해당하는 values를 가져오기)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.get("ENTJ")
```

```
>>> 대담한 통솔자
```

```
mbti_dict.get("ESFP")
```

```
>>> None
```

Key가 없어도 에러가 발생하지 않음!

1-5. 딕셔너리 자료형



- 딕셔너리 관련 함수 – **in** (해당 key가 딕셔너리에 있는지 확인)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
"INFP" in mbti_dict  
>>> True
```

```
"ESFP" in mbti_dict  
>>> False
```


1-5. 딕셔너리 자료형



- 딕셔너리 관련 함수 – **pop** (key에 해당하는 value를 꺼내면서 삭제)

```
mbti_dict = {  
    "ENFP": "재기발랄한 활동가",  
    "INFP": "열정적인 중재자",  
    "ENTJ": "대담한 통솔자",  
    "INTP": "논리적인 사색가"  
}
```

```
mbti_dict.pop("INTP")  
>>> '논리적인 사색가'
```

```
mbti_dict  
>>> {'ENFP': '재기발랄한 활동가', 'INFP': '열정적인 중재자', 'ENTJ': '대담한 통솔자'}
```



■ 집합 구조

```
s1 = set([1, 2, 3])    # set 키워드로 만들 수 있음  
s1  
>>> {1, 2, 3}
```

■ 집합 자료형의 특징

- 중복을 허용하지 않음
- 순서가 없음 (=인덱스 접근 불가)
- set()의 괄호() 안에 리스트를 입력하거나 문자열 입력 가능
- 리스트 변환 시 중복 제거 목적으로 활용 가능함



■ 집합 예시

```
s2 = set('hello')    # l이 2개 중복됨
```

```
s2
```

```
>>> {'e', 'h', 'l', 'o'}    # 순서도 바뀌고, 중복된 문자 'l' 하나가 제거됨
```

```
s3 = {'d', 'c', 'b', 'a'}
```

```
s3
```

```
>>> {'a', 'b', 'c', 'd'}    # 집합은 순서의 개념이 없어서 순서가 바뀌기도 함
```

집합 자료형은 순서가 없기 때문에 인덱싱으로 요솟값을 얻을 수가 없는 특징이 있음

1-6. 집합 자료형



```
s1 = set([1, 2, 3, 4, 5, 6])  
s2 = set([4, 5, 6, 7, 8, 9])
```

■ 교집합

```
s1 & s2  
>>> {4, 5, 6}
```

```
s1.intersection(s2)  
>>> {4, 5, 6}
```

■ 합집합

```
s1 | s2  
>>> {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
s1.union(s2)  
>>> {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

■ 차집합

```
s1 - s2  
>>> {1, 2, 3}
```

```
s1.difference(s2)  
>>> {1, 2, 3}
```

1-6. 집합 자료형



■ 값 1개 추가하기 - add

```
s1 = set([1, 2, 3])  
s1.add(4)  
s1  
>>> {1, 2, 3, 4}
```

■ 값 여러개 추가하기 - update

```
s1 = set([1, 2, 3])  
s1.update([4, 5, 6])  
s1  
>>> {1, 2, 3, 4, 5, 6}
```

■ 값 제거하기 - remove

```
s1 = set([1, 2, 3])  
s1.remove(2)  
s1  
>>> {1, 3}
```

■ 값 제거하기 - discard

```
s1 = set([1, 2, 3])  
s1.discard(4)  
s1  
>>> {1, 2, 3}
```

remove와 discard 함수는 비슷하지만,
discard는 존재하지 않는 값을 제거하려 해도 오류가 발생하지 않음



- 불(bool) 자료형
 - 참(True)
 - 거짓(False)

```
a = True  
print(type(a))  
>>> <class 'bool'>
```

첫 글자는 항상 대문자로 작성

```
1 == 1  
>>> True
```

조건문의 반환값으로도 사용

```
2 < 1  
>>> False
```

1-7. 불 자료형



문자열

"python"	True
" "	False

딕셔너리

{ 'a':1 }	True
{ }	False

리스트

[1,2,3]	True
[]	False

숫자

1	True
0	False

튜플

(1,2,3)	True
()	False

참고

None	False
------	-------



■ 불(bool) 연산

```
a = bool('python')  
print(a)  
>>> True
```

```
a = bool('')  
print(a)  
>>> False
```

```
a = bool([1, 2, 3])  
print(a)  
>>> True
```




■ 논리 연산자

■ and 연산자

```
True and True  
>>> True
```

```
True and False  
>>> False
```

```
False and True  
>>> False
```

```
False and False  
>>> False
```

■ or 연산자

```
True or True  
>>> True
```

```
True or False  
>>> True
```

```
False or True  
>>> True
```

```
False or False  
>>> False
```

■ not 연산자

```
not True  
>>> False
```

```
not False  
>>> True
```

```
not 1  
>>> False
```

```
not 0  
>>> True
```

1-8. 변수와 메모리



■ 변수

- 객체를 가리키는 것
- id는 변수가 가리키는 객체의 주소 값을 반환



```
a = [1, 2, 3] # 변수a는 리스트가 저장된 메모리 주소를 가리킴  
id(a)  
>>> 4303029896
```

```
a, b = ('python', 'life') # 튜플 변수 생성  
(a, b) = 'python', 'life'
```

```
[a, b] = ['python', 'life'] # 리스트 변수 생성
```

```
a = b = 'python' # 여러 변수에 같은 값 대입 가능
```

```
a, b = b, a # 두 변수 값 바꾸기
```

1-8. 변수와 메모리



■ 변수 생성

- =(assignment) 기호 사용
- 변수_이름 = 변수에_저장할_값

■ 변수명 규칙

- 영문자, 숫자, 언더스코어(_)만 사용
- 숫자로 시작할 수 없다.
- 예약어는 사용할 수 없다.
- 대소문자를 구분한다.

올바른 변수명 예시

```
name = "홍길동"
age = 25
user_name = "gildong"
userName = "gildong"
_private = "비공개"
count1 = 10
```

잘못된 변수명 예시

```
1name = "홍길동"      # 숫자로 시작 (오류)
user-name = "홍길동"  # 하이픈 사용 (오류)
if = 10               # 예약어 사용 (오류)
```

파이썬 예약어 예시

False, None, True, and, not, or, for,
break, if, class, continue, def, del 등

■ 리스트 복사

■ 참조 복사

```
a = [1, 2, 3]
b = a
a[1] = 4
```

```
print(a)
>>> [1, 4, 3]
```

```
print(b)
>>> [1, 4, 3]
```

a, b 주소값 같음

■ [:] 이용하기

```
a = [1, 2, 3]
b = a[:]
a[1] = 4
```

```
print(a)
>>> [1, 4, 3]
```

```
print(b)
>>> [1, 2, 3]
```

a, b 주소값 다름

■ copy 함수 이용하기

```
a = [1, 2, 3]
b = a.copy()
```

```
print(a)
>>> [1, 2, 3]
```

```
print(b)
>>> [1, 2, 3]
```

a, b 주소값 다름



■ 중첩 리스트와 깊은 복사

■ shallow copy

```
a = [[1, 2], [3, 4]]
b = a.copy()

a[0][0] = 99

print(a)
# [[99, 2], [3, 4]]

print(b)
# [[99, 2], [3, 4]]
# 내부 리스트를 공유하여 함께 변경됨
```

■ deep copy

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)

a[0][0] = 99

print(a)
# [[99, 2], [3, 4]]

print(b)
# [[1, 2], [3, 4]]
```